ACTIVITY 3.2 · UNIT 3 — PROGRAMMING & AUTOMATION

Behaviors & Pseudocode

Student Edition — VEX EXP · C++ in VS Code

Name: _____ **Date:** _____ **Period:** _____

Introduction

This is the **planning half of programming** — how to turn a goal into instructions a robot can follow, before you touch a keyboard. You'll meet **behaviors** (the things a robot does) and the single most useful skill in programming: **breaking a complex behavior down** into simple ones, and those into basic single commands. Then you'll write **pseudocode** — a plain-language plan, written in order, before you worry about exact syntax — and see how a few lines of that plan become real **VEX C++**. The "sense, decide, act" loop from Chapter 8 shows up the moment you write your first **if** or **while**. You write real code on the robot starting in Chapter 10. Read Chapter 9 in the textbook for the full explanation.

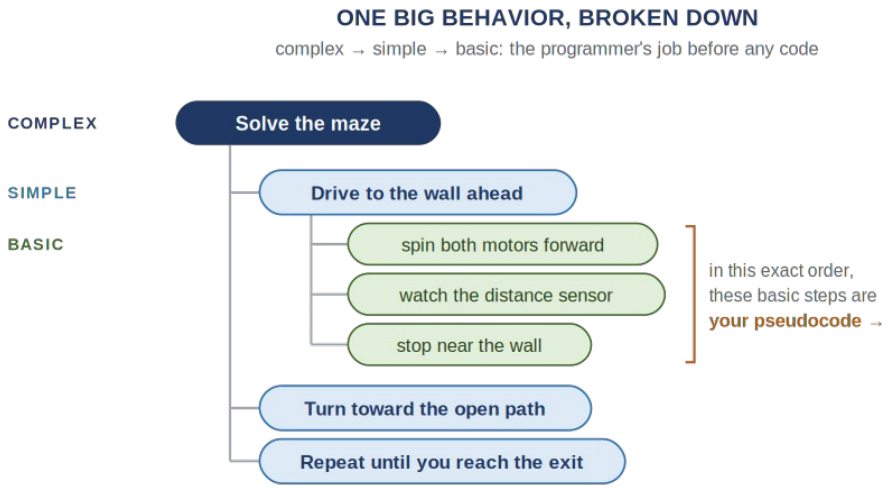
Learning Objectives

- Define a behavior, and classify behaviors as basic, simple, or complex.
- Break a complex behavior down into simple and then basic behaviors (decomposition).
- Write clear pseudocode for a robot task — in order, behavior-focused, with blocks marked by curly braces { }.
- Explain the programmer's role versus the robot's, and connect an if/while behavior to the sense → decide → act loop from Chapter 8.

Key Ideas (quick reference)

- **Three sizes of behavior:** basic (a single command — about one line of code), simple (a few basic behaviors that do something useful — the sweet spot), and complex (a big job, like solving a maze).
- **Decomposition is the key skill:** a complex behavior can always be broken into simpler ones, and those into basic ones. Keep splitting until every piece is something the robot can do directly.
- **Abstraction is decomposition run upward:** wrap a group of small steps under one name, then work with that name instead of the steps inside it. "Brush my teeth" is an abstraction; in code, a function names a group of steps (Ch12) and an object (like a smart drivetrain) names a bundle of parts plus what you can tell it to do (Ch13).
- **Pseudocode is the plan before the code:** a plain-language outline written in the same order the program will run. Express what each step does, not perfect syntax (keep it reasonable). Curly braces { } mark a block — steps that belong together.
- **Programmer vs. robot:** you plan completely and in order; the robot carries it out literally, with no common sense to fill gaps. Any behavior with an if or a while is the Chapter 8 feedback loop — sense, decide, act — in disguise.

Breaking a Behavior Down



A complex behavior ("solve the maze") breaks into simple behaviors, and one of those breaks into basic single commands. Read the bottom layer in order and you have your pseudocode.

Breaking Down, and Building Up

Decomposition runs one way — big into smaller pieces. Its partner, **abstraction**, runs the other way: take a handful of small steps, give them **one name**, and from then on work with that name instead of the steps inside it. "Brush my teeth" is an abstraction — one name standing for a dozen small steps; you say the name and skip the details, which lets you think in bigger steps.

Read the tree above both ways: top-down is decomposition; bottom-up is abstraction — a few basic commands become one named behavior ("drive to the wall"), and a few of those become one bigger name ("solve the maze"). You'll meet abstraction twice more in code: a **function** names a group of steps (Chapter 12), and an **object** — like the smart drivetrain in Chapter 13 — names a bundle of parts and the things you can tell it to do.

From Plan to Code

Here is the "drive to the wall" behavior as pseudocode, then the same plan as real VEX C++. Notice they line up almost line-for-line — and the curly braces { } do the same job in both: they mark a block of steps that belong together. (You don't need to understand every C++ symbol yet — that's Chapter 10.)

Pseudocode — the plan

```
/*
  behavior: drive to the wall ahead, then stop
  {
      spin both drive motors forward
      while the distance sensor reads more than 200 mm
      {
          keep rolling (wait a moment, then check again)
      }
      stop both drive motors
  }
*/
```

VEX C++ — the real thing (Chapter 10)

```
// behavior: drive to the wall ahead, then stop
leftMotor.spin(forward);
```

```
rightMotor.spin(forward);

while (distance.objectDistance(mm) > 200) {
    wait(20, msec);
}

leftMotor.stop();
rightMotor.stop();
```

What You'll Need

- Your engineering notebook and a pencil
- The behavior-and-pseudocode worksheet (this packet)
- (Optional) your robot from Chapter 8 on the table, to picture the behaviors
- No computer or code yet — this chapter is planning

Procedure

1. **Break down the maze.** Using the decomposition tree above as a model, take one complex behavior and break it into simple behaviors, then break one of those into basic single commands. Stop when every bottom item is something the robot could do directly.
2. **Decompose the challenge task.** For the task in Worksheet 1 — "when a pushbutton is pressed, the robot drives as fast as possible for 20 feet (about 10 seconds), then stops" — fill in the three-column chart: the complex behavior, its simple behaviors, and the basic behaviors.
3. **Write the pseudocode.** In Worksheet 2, turn your basic behaviors into pseudocode, in order, using the /* { } */ template. Mark the blocks that belong together with curly braces.
4. **Decompose an everyday behavior.** In Worksheet 3, list at least five smaller behaviors inside "brushing my teeth," and notice how some of those could break down even further.
5. **Test your plan on a human.** Trade pseudocode with a partner and have them follow it literally — doing exactly what each line says and nothing more. If they end up somewhere you didn't intend, your plan has a gap. Fix it. (That gap is exactly what the robot would have tripped on.)

Worksheet 1 — Break the task into behaviors

The task: **when a pushbutton is pressed, the robot drives as fast as possible for 20 feet (about 10 seconds), then stops.** Fill in the chart — the complex behavior, the simple behaviors it breaks into, and the basic single-command behaviors.

Complex behavior	Simple behaviors	Basic behaviors (≈ one line of code)

Worksheet 2 — Write the pseudocode

Write pseudocode for that same task, **in the order it will run**. Use the template below as a starting frame — describe what each step does, and use curly braces { } to mark the block of steps that belong together.

Template

```
/*
{









}
*/
```

Worksheet 3 — Decompose an everyday behavior

Break the complex behavior **"brushing my teeth"** into at least five smaller behaviors. (Could any of them break down even further?)

List 5+ smaller behaviors inside "brushing my teeth"

Conclusion Questions

Answer in complete sentences.

1. Why is it useful to think of a program as a set of basic, simple, and complex behaviors?
2. What does a pair of curly braces { } mark in a C++ program?
3. What is the role of a programmer, and how is it different from the robot's role?
4. Write this plan as pseudocode: "drive forward until the distance sensor reads less than 150 mm, then stop." Which part is the sensing, which is the deciding, and which is the acting?
5. Why write pseudocode before real code — what does planning on paper save you?
6. A complex behavior is "sort the parts by color." Name two simple behaviors it might break into, and one basic behavior inside one of those.

7. Decomposition breaks a big behavior down into smaller ones. What is abstraction, and how is "brush my teeth" an example of it?
